

РК 2

РТ5-61Б Слкуни Герман

Вариант 19 (18, т.к. файл 19 варианта битый)

1. Загрузка и очистка данных

```
In [3]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import numpy as np
```

```
In [4]: df = pd.read_csv('WHI_Inflation.csv', encoding='utf-8')
df.head(10)
```

```
Out[4]:
```

	Country	Year	Headline Consumer Price Inflation	Energy Consumer Price Inflation	Food Consumer Price Inflation	Official Core Consumer Price Inflation	Producer Price Inflation
0	Afghanistan	2015	-0.660000	-4.250000	-0.840000	0.219999	NaN
1	Afghanistan	2016	4.380000	2.070000	5.670000	5.192760	NaN
2	Afghanistan	2017	4.976000	4.440000	6.940000	5.423228	NaN
3	Afghanistan	2018	0.630000	1.474185	-1.045952	-0.126033	NaN
4	Afghanistan	2019	2.302000	-2.494359	3.794770	NaN	NaN
5	Afghanistan	2020	5.443000	NaN	5.829005	NaN	NaN
6	Afghanistan	2021	5.062000	NaN	NaN	NaN	NaN
7	Afghanistan	2022	10.600000	NaN	NaN	NaN	NaN
8	Afghanistan	2023	NaN	NaN	NaN	NaN	NaN
9	Albania	2015	1.910179	-0.520000	4.319489	-0.156957	NaN

```
In [5]: # Целевая переменная
target = 'Headline Consumer Price Inflation'
```

```
df.dropna(subset=[target], inplace=True)
df.head(10)
```

Out[5]:

	Country	Year	Headline Consumer Price Inflation	Energy Consumer Price Inflation	Food Consumer Price Inflation	Official Core Consumer Price Inflation	Producer Price Inflation
0	Afghanistan	2015	-0.660000	-4.250000	-0.840000	0.219999	NaN
1	Afghanistan	2016	4.380000	2.070000	5.670000	5.192760	NaN
2	Afghanistan	2017	4.976000	4.440000	6.940000	5.423228	NaN
3	Afghanistan	2018	0.630000	1.474185	-1.045952	-0.126033	NaN
4	Afghanistan	2019	2.302000	-2.494359	3.794770	NaN	NaN
5	Afghanistan	2020	5.443000	NaN	5.829005	NaN	NaN
6	Afghanistan	2021	5.062000	NaN	NaN	NaN	NaN
7	Afghanistan	2022	10.600000	NaN	NaN	NaN	NaN
9	Albania	2015	1.910179	-0.520000	4.319489	-0.156957	NaN
10	Albania	2016	1.291234	0.040000	3.249188	0.151382	-1.613304

```
In [6]: # Удалим Official Core CPI, ибо может пересекаться с Headline CPI и мешать.
X = df.drop(columns=[target, 'Official Core Consumer Price Inflation'])
y = df[target]
```

```
In [7]: # Определение числовых и категориальных признаков
numerical_features = X.select_dtypes(include=np.number).columns.tolist()
categorical_features = X.select_dtypes(include='object').columns.tolist()
```

```
In [8]: # Числовые признаки: заполнение пропусков медианой
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median'))
])

# Категориальные признаки: заполнение пропусков (mode) и One-Hot Encoding
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Объединение пайплайнов в ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ]
)
```

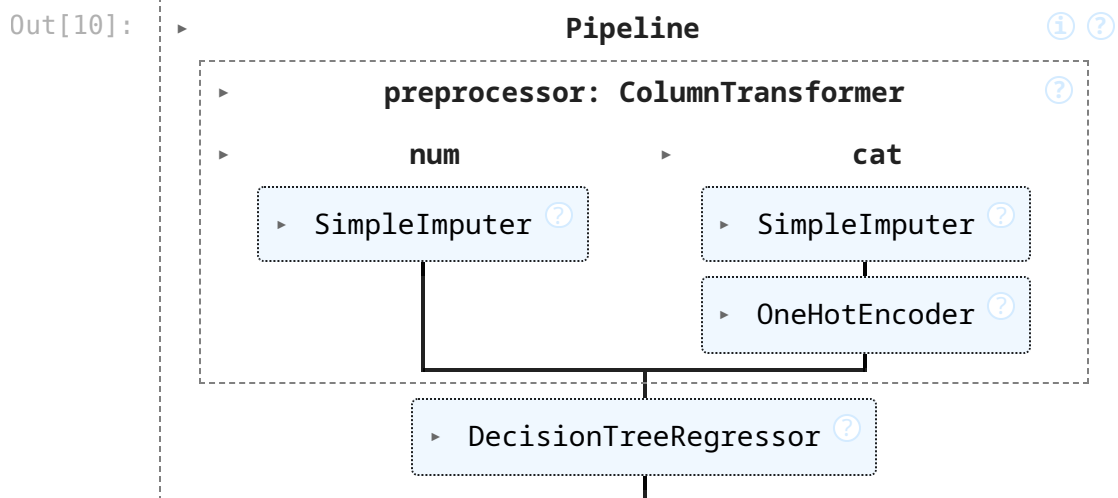
```
In [9]: # Разделение данных на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Размер обучающей выборки X:", X_train.shape)
print("Размер тестовой выборки X:", X_test.shape)
print("Размер обучающей выборки y:", y_train.shape)
print("Размер тестовой выборки y:", y_test.shape)
```

```
Размер обучающей выборки X: (960, 14)
Размер тестовой выборки X: (240, 14)
Размер обучающей выборки y: (960,)
Размер тестовой выборки y: (240,)
```

2. Построение и оценка моделей

```
In [10]: # Создание и обучение модели Древа решений
dt_model = Pipeline(steps=[('preprocessor', preprocessor),
                             ('regressor', DecisionTreeRegressor(random_state=42))])
dt_model.fit(X_train, y_train)
```



```
In [12]: # Прогнозирование на тестовой выборке
y_pred_dt = dt_model.predict(X_test)

# Оценка качества модели
mae_dt = mean_absolute_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))
r2_dt = r2_score(y_test, y_pred_dt)

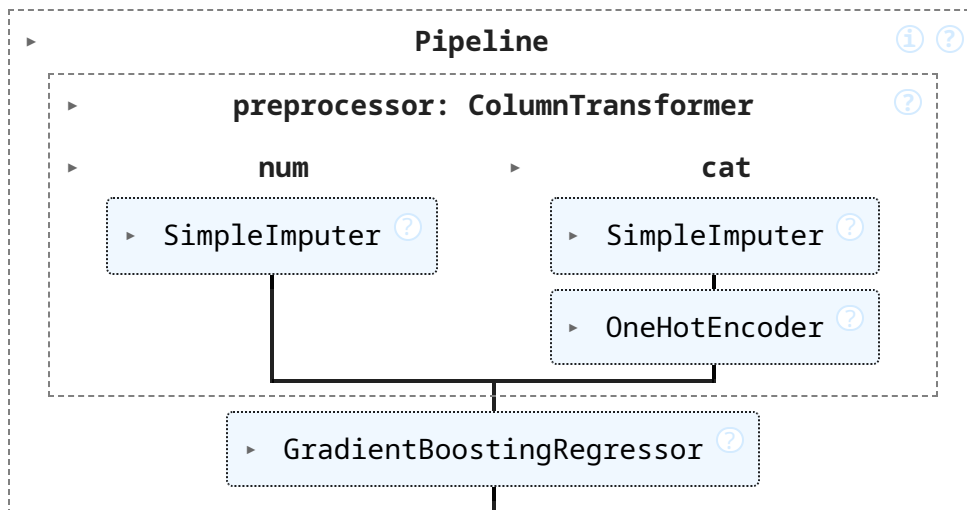
print("Метрики дерева решений")
print(f"MAE: {mae_dt:.3f}")
print(f"RMSE: {rmse_dt:.3f}")
print(f"R2 Score: {r2_dt:.3f}")
```

```
Метрики дерева решений
MAE: 2.942
RMSE: 9.524
R2 Score: 0.214
```

```
In [13]: # Создание и обучение модели Градиентного бустинга
gb_model = Pipeline(steps=[('preprocessor', preprocessor),
                           ('regressor', GradientBoostingRegressor(random_st

gb_model.fit(X_train, y_train)
```

Out[13]:



```
In [14]: # Прогнозирование на тестовой выборке
y_pred_gb = gb_model.predict(X_test)

# Оценка качества модели
mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
r2_gb = r2_score(y_test, y_pred_gb)

print("Метрики градиентного бустинга:")
print(f"MAE: {mae_gb:.3f}")
print(f"RMSE: {rmse_gb:.3f}")
print(f"R2 Score: {r2_gb:.3f}")
```

Метрики градиентного бустинга:

MAE: 2.246

RMSE: 7.970

R2 Score: 0.450

Выводы

Градиентный бустинг показывает себя лучше, чем дерево решений.

In []: